

Potentially Visible Set Generation with the Disocclusion Buffer

SEBASTIAN KÜNZEL, University of Stuttgart, Germany
SERGEJ GERINGER, University of Stuttgart, Germany
QUYNH QUANG NGO, University of Stuttgart, Germany
PHILIP VOGLREITER, Graz University of Technology, Austria
DANIEL WEISKOPF, University of Stuttgart, Germany
DIETER SCHMALSTIEG, University of Stuttgart, Germany

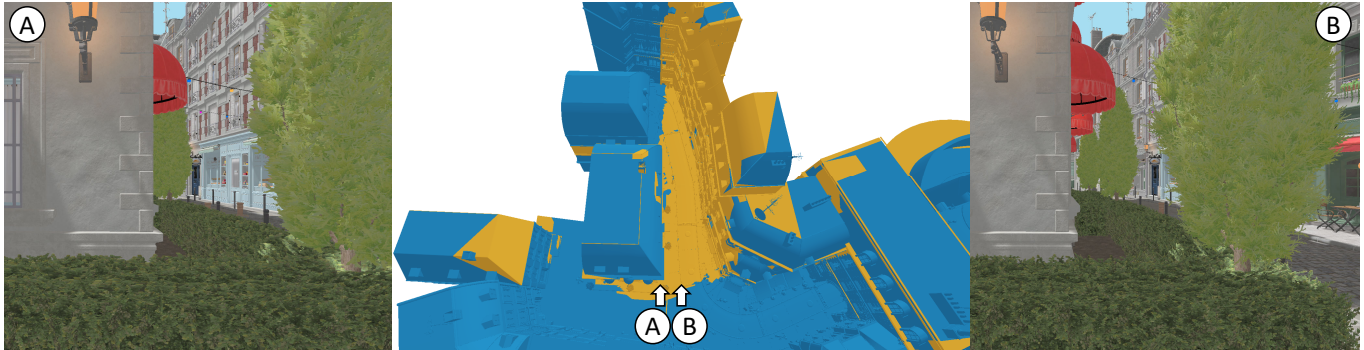


Fig. 1. Two views, A and B, seen from the street level in the Bistro Exterior scene [Amazon Lumberyard 2017]. The middle image shows the potentially visible set (yellow parts of the scene) computed using our new method. The potentially visible set is determined for a viewcell centered at viewpoint A and extending to include both viewpoints A and B. Note that the trees disoccluded in view B are correctly included in the potentially visible set.

The computation of a potentially visible set (PVS) can accelerate many computer graphics algorithms, such as framerate upsampling, streaming rendering, global illumination, and multi-fragment effects. Algorithms for from-region PVS have an inherently high complexity. Previous from-region PVS algorithms propagate occlusion through the scene in a front-to-back manner and are order-dependent, which places bounds on parallelism and restricts execution speed. We introduce the disocclusion buffer, which operates on a sparse, layered representation of the scene with quantized depth. In this representation, we invert the traditional PVS problem formulation and explicitly compute disocclusion rather than occlusion. Disocclusion can be computed in parallel in an order-independent manner, overcoming the main bottleneck in traditional PVS computation. Our PVS algorithm is over six times faster than the previous state of the art at the same level of accuracy in a direct comparison. It runs in shaders on the GPU without requiring any hardware extensions. We demonstrate how our work outperforms previous PVS algorithms in the range of supported camera motion without compromising quality.

Authors' Contact Information: Sebastian Künzle, University of Stuttgart, Stuttgart, Germany, sebastian.kuenzel@visus.uni-stuttgart.de; Sergej Geringer, University of Stuttgart, Stuttgart, Germany, sergej.geringer@visus.uni-stuttgart.de; Quynh Quang Ngo, University of Stuttgart, Stuttgart, Germany, quynh.ngo@visus.uni-stuttgart.de; Philip Voglreiter, Graz University of Technology, Graz, Austria, voglreiter@icg.tugraz.at; Daniel Weiskopf, University of Stuttgart, Stuttgart, Germany, daniel.weiskopf@visus.uni-stuttgart.de; Dieter Schmalstieg, University of Stuttgart, Stuttgart, Germany, dieter.schmalstieg@visus.uni-stuttgart.de.



This work is licensed under a Creative Commons Attribution 4.0 International License.
SA Conference Papers '25, Hong Kong, Hong Kong
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2137-3/2025/12
<https://doi.org/10.1145/3757377.3763981>

CCS Concepts: • **Computing methodologies** → **Rasterization**; **Visibility**.

Additional Key Words and Phrases: Potentially visible set, disocclusion buffer, real-time rendering

ACM Reference Format:

Sebastian Künzle, Sergej Geringer, Quynh Quang Ngo, Philip Voglreiter, Daniel Weiskopf, and Dieter Schmalstieg. 2025. Potentially Visible Set Generation with the Disocclusion Buffer. In *SIGGRAPH Asia 2025 Conference Papers (SA Conference Papers '25)*, December 15–18, 2025, Hong Kong, Hong Kong. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3757377.3763981>

1 Introduction

The computation of visibility has many applications in computer graphics, such as computing shadows and global illumination. Another use of visibility is to reduce the shading load by occlusion culling [Cohen-Or et al. 2003]. Parts of the scene classified as occluded are culled before shading for improved rendering speed and stable frame rates.

Determining visibility exactly is hard [Durand et al. 2002], but not necessary to perform effective occlusion culling. Instead of exact visibility, occlusion culling can use a *potentially visible set* (PVS), which can be efficiently computed at the expense of some visibility overestimation [Airey et al. 1990]. The simplest form of PVS is a from-point PVS, but it is only valid for one camera position. In contrast, a from-region PVS is valid for a whole region (or *viewcell*) in which the camera can be placed freely. Computing the PVS for a region means that it remains valid as long as the camera does not leave the viewcell [Wonka et al. 2001]. Under such a spatiotemporally coherent camera movement, various applications can benefit from

re-using visibility over multiple frames. Popular use cases include framerate upsampling and streaming rendering [Hladky et al. 2019a; Mueller et al. 2018], global illumination [Popescu et al. 2024] and multi-fragment effects, such as defocus blur or order-independent transparency [Vasilakis et al. 2020].

Game engines often provide tools for partitioning a static scene into regions and precomputing a PVS for each region [Epic Games 2022]. However, this approach requires significant storage and is limited to scenes known in advance. Therefore, recent research investigated the online computation of the PVS. These techniques rely on hardware raytracing [Koch and Wimmer 2021], rendering the scene multiple times [Kim and Lee 2023; Mueller et al. 2018] or depth sorting of the scene [Hladky et al. 2019a; Voglreiter et al. 2023]. Raytracing and repeated rendering become too expensive for high image resolution and complex geometry, while front-to-back traversal must progress in synchronized steps, making it difficult to keep hardware pipelines filled [Mattausch et al. 2008].

In this paper, we present a novel approach for the online computation of a from-region PVS. Our method creates a *disocclusion buffer*, a multi-fragment representation that explicitly stores and propagates which parts of the scene are visible. Our method has the advantage that it does not require any explicit synchronization. The disocclusion buffer is defined as a regular grid in normalized device coordinates that stores visibility information for each grid cell. After initializing the disocclusion buffer with the scene geometry using a single rasterization pass, we inspect all cells in parallel and propagate disocclusion caused by empty cells to the affected part of the scene. The final visibility per cell can be efficiently queried using bit operations. This paper presents the following contributions:

- We introduce the disocclusion buffer, a novel data structure for efficiently computing a PVS. Our method runs in compute shaders and requires neither depth sorting nor synchronized traversal between layers. It uses only a single rasterization pass, making it suitable for dynamic and very large scenes.
- As a secondary contribution, we introduce an efficient volumetric filter that drastically reduces approximation errors in the PVS even for problematic geometry, such as tiny or slithery triangles.

We present experimental results characterizing the performance of our method in terms of speed, memory consumption and PVS quality. In direct comparison, the disocclusion buffer method is about six times faster than the previous fastest PVS method [Voglreiter et al. 2023].

2 Related work

We discuss relevant work organized into two areas, visibility computation and multi-fragment representations.

2.1 Visibility computation from a point

The goal of occlusion culling is to quickly identify parts of the scene that are not visible. A from-point PVS can be computed on the fly. For example, a visibility buffer [Burns and Hunt 2013] captures depth and primitive ID information of visible pixels and forwards them to the shading stage. However, for scenes with high geometric complexity, it may pay off to generate a from-point PVS [Bittner and Wonka 2003] before shading.

A from-point PVS stores the part of the scene contained in an occlusion volume, conceptually similar to a shadow volume [Hudson et al. 1997]. An occlusion volume can be constructed using a BSP tree [Bittner et al. 1998], an octree [Schauffler et al. 2000], a hierarchical depth buffer [Greene et al. 1993] or a stack of object coverage masks [Durand et al. 2000]. A key requirement for efficient PVS computation is the fusion of many small occluders into the occlusion volume, through rasterization methods [Hasselgren et al. 2016; Hudson et al. 1997; Leyvand et al. 2003; Wonka et al. 2000] or by finding virtual occluders fully contained in the occlusion volume [Chandak et al. 2009; Koltun et al. 2000]. High-quality virtual occluders typically require offline processing [Tan et al. 2025].

PVS algorithms are often classified into conservative and aggressive methods [Hudson et al. 1997]. Conservative algorithms do not allow any false negatives (visible triangles incorrectly excluded from the PVS). Aggressive algorithms accept a small amount of false negatives in favor of reducing false positives (occluded triangles unnecessarily included in the PVS). In general, false positives are less problematic than false negatives, as the former cannot lead to image errors. However, Wonka et al. [Bittner et al. 2009] point out that PVS algorithms can typically only be conservative from a theoretical point of view. In practice, numerical computation always operates with limited precision and can incorrectly classify spurious geometry as visible or occluded. Such errors must be expected, since the geometry in real-world scenes is often poorly conditioned. Voglreiter et al. [Voglreiter et al. 2023] define unreliable triangles as “tiny or slithery triangles” that, “after projection to image space, have a size of less than one pixel along any of their three edges”. Typically, such triangles have negligible impact on image quality.

Several authors [Bittner et al. 2009; Koch and Wimmer 2021; Nirenstein and Blake 2004; Voglreiter et al. 2023] suggest counting pixel instead of triangle errors to characterize PVS quality. (Figure 7) highlights pixels that contain at least one slithery triangle. A pixel with a triangle mismatch may result from multiple slithery triangles in close proximity depth or coverage fighting. Our proposed volumetric filtering significantly reduces this type of error.

2.2 Visibility computation from a region

More advanced applications aim at reusing a PVS in space (e.g., stereoscopic rendering, defocus effects) or time (e.g., reuse of shading or indirect illumination). In this case, a from-point PVS is not sufficient, and a from-region PVS is needed. Such a PVS is computed for a viewcell, i.e., a region defined in 4D or even 5D ray-space [Durand et al. 2002]. Naturally, the algorithmic complexity of from-region visibility, a 4D or 5D problem, is significantly higher than the one of from-point visibility in 2D ray-space.

A simple approach partitions a scene into discrete viewcells and precomputes a from-region PVS for each viewcell [Teller and Séquin 1991]. This approach is commonly used by commercial solutions, such as the Unreal Engine [Epic Games 2022]. Unfortunately, an approach based on precomputation requires additional storage [Serpa and Rodrigues 2019] and cannot accommodate dynamic scenes. Some recent research therefore computes a from-region PVS online, either by repeated rasterization from multiple future viewpoints assumed close to the current camera position [Hladky et al. 2022;

Kravec et al. 2023; Mueller et al. 2018; Reinert et al. 2016] or by tracing rays through the scene [Koch and Wimmer 2021; Popov et al. 2013]. Neither rasterization nor raytracing scales well with increasing image resolution, scene complexity or viewcell size.

Similar limitations apply to methods that traverse a depth-sorted scene in front-to-back order. Sorted traversal is required to ensure that occluder fusion works efficiently, as occlusion effects of early occluders close to the camera propagate to parts of the scene farther away. However, full sorting of fragments can be very costly. For example, Hladky et al. [Hladky et al. 2019a] generate visibility samples in the form of per-pixel linked lists at a drastically subsampled resolution of 200×100 to achieve real-time performance.

As an alternative to full sorting, a scene can be represented by hierarchically grouping primitives and applying approximate sorting to primitive groups rather than individual primitives or fragments. In this case, front-to-back order requires the rendering of primitive groups closer to the camera to conclude before more distant primitive groups may be traversed. In turn, GPU-wide synchronization must be enforced after each step. This synchronization flushes the pipeline and reduces throughput [Haar and Aaltonen 2015].

The need for explicit synchronization can be reduced by intelligent scheduling and buffering. For example, hardware occlusion queries can be buffered to avoid latency [Bittner et al. 2004; Mattausch et al. 2008]. Voglreiter et al. [Voglreiter et al. 2023] peel layers off an octree. However, their framerates are ultimately limited by the need to establish GPU-wide synchronization after each octree layer. Kim et al. [Kim and Lee 2023] implement front-to-back traversal by depth peeling, which requires repeated rasterization of the scene. To obtain real-time frame rates, they restrict their implementation to peeling only two layers and resolve the remaining visibility using a heuristic method. To further increase throughput, visibility per fragment is only approximated by stochastic sampling.

Our approach avoids the limitations mentioned above: It performs only a single rasterization pass and computes the PVS without any need for a sorted scene. Our algorithm can occupy the entire GPU and is limited only by memory transfer rates.

2.3 Multi-fragment representations

Various computer graphics techniques rely on a sampled representation of a scene as an intermediate data structure. Processing the scene primitives directly often does not scale well with increasing scene complexity. Therefore, many techniques, such as deferred rendering [Saito and Takahashi 1990], scan-convert the scene through rasterization and continue working only on the visible pixels. Such screen-space techniques can be seen as simple instances of decoupled sampling [Ragan-Kelley et al. 2011], which describes the idea of separating the spatiotemporal sampling frequencies of visibility and shading in image synthesis.

If a flat buffer is insufficient for the desired effect, a *multi-fragment representation* can be used to store more than one sample per pixel position. Vasilakis et al. [Vasilakis et al. 2020] classify these representations into three groups: A-buffer [Carpenter 1984], depth peeling [Everitt 2001] and k -buffer [Bavoil et al. 2007].

The A-buffer stores all samples that fall onto a pixel position, often as a linked list [Yang et al. 2010]. It is commonly mentioned

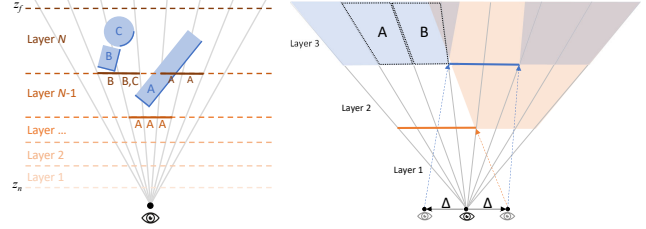


Fig. 2. (left) Object A maps to three tiles of layer $N - 1$ and two tiles of layer N , whereas objects B and C are fused in one of the tiles of layer N . (right) If we allow a viewpoint movement by up to Δ , the area marked in orange is disoccluded in layer 1, while the area marked in blue is disoccluded in layer 2. In layer 3, only the area disoccluded both by layer 1 and layer 2 must be included in the PVS. In particular, we can determine that tile B is disoccluded, but tile A is occluded because it is not visible from layer 1.

as the gold standard for antialiasing and order-independent transparency in production rendering. Another use case are layered depth images [Shade et al. 1998], which support novel view synthesis. The main drawbacks of the A-buffer are its unbounded memory requirements and its lack of support for random access of individual fragments. Depth-sorting the individual lists can be very expensive [Voglreiter et al. 2023].

Depth peeling processes the fragments in an iterated rendering pipeline, implying repeated depth testing. While depth peeling has very small memory requirements, its runtime is proportional to the number of peeling iterations, making it too slow for most real-time applications [Vasilakis et al. 2020]. Depth peeling has been used for applications such as order-independent transparency [Everitt 2001] or screen-space global illumination [Mara et al. 2016]. However, many applications are limited to only a very small number of iterations, such as $k = 2$ [Kim and Lee 2023; Mara et al. 2016].

A k -buffer offers a middle ground by storing a maximum of k samples per fragment. Typically, only a single rendering pass is required, and a k -buffer is more memory-efficient than an A-buffer. Moreover, it allows one to strategically select the k -best samples according to an application-specific metric. A typical metric is to keep the k fragments closest to the camera [Bavoil et al. 2007]. Our disocclusion buffer can be classified as a variant of a k -buffer with a sparse memory layout and a metric that retains the closest fragments in each of k non-overlapping depth intervals. We will show how this representation lends itself to an extremely efficient PVS computation.

3 Method

In a nutshell, our disocclusion buffer is a multi-fragment extension of a visibility buffer [Burns and Hunt 2013]. Like a visibility buffer, the disocclusion buffer stores depth and ID attachments, but it is neither flat nor dense. We organize fragments into *tiles*, i.e., rectangular groups of samples, sparsely located in an array of planes parallel to the image plane. We call the slab between two planes a *layer*. The tiles in the disocclusion buffer represent the scene geometry, but resampled into a sparse regular grid in normalized device coordinates. We first explain how to generate the disocclusion buffer, then describe how to use the disocclusion buffer to compute a PVS.

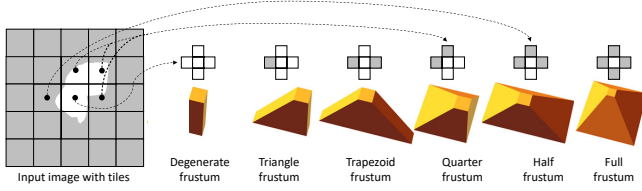


Fig. 3. (left) A grid of 4×4 tiles partially covered by scene geometry (gray). We consider the occupation status of neighboring tiles (gray/white pixels) for disocclusion computations in the scene. Partially filled or empty tiles are classified as “open”, whereas tiles completely filled with samples (gray) are “closed.” (right) Patterns of edges between open and closed tiles lead to unique disocclusion frusta (up to rotation).

3.1 Disocclusion buffer generation

To build the disocclusion buffer, we rasterize the scene from the current viewpoint and write primitive ID and depth per fragment into the disocclusion buffer. We retain separate depth buffers for each layer. From two fragments in the same layer that map to the same location in image space, the depth test retains only the closer one. This rule ensures that the fragment closest to the viewpoint is always present in the disocclusion buffer, although it may not be found in the frontmost layer. Fragments hidden from the current viewpoint may be contained in the disocclusion buffer only if they fall into different layers (see Figure 2, left). Fragments of primitives that span two layers will be assigned either to a tile in the first layer or to a tile in the second layer, depending on their depth values.

3.2 PVS generation

The PVS computation makes use of the fact that, after sampling and quantizing the scene geometry to one entry per layer and pixel location, disocclusions can occur only between layers. This observation is the key to implementing a drastically simplified PVS algorithm. Unlike other PVS algorithms that must propagate occlusions front-to-back through the scene, we can explicitly and simultaneously compute *disocclusions* anywhere in the scene. A critical advantage of our method is its ability to work without access to geometric primitives and in an order-independent manner.

The PVS computation operates at the granularity of tiles. Assume two tiles of the same layer, but in different image locations (Figure 2, right): A is disoccluded from layer 2, but not from layer 1. B is disoccluded from both layers. For inclusion in the PVS, a tile must be disoccluded in *all* previous layers, which applies only to B.

Our goal is to efficiently compute disocclusion for all tiles and all previous layers. We determine whether any view ray emitted from within the viewcell can hit the tile without being blocked by an earlier layer. An opening in a previous layer causes the area behind it to be disoccluded. More specifically, a tile-shaped opening creates a frustum expanding towards the back of the scene. If a tile is in the volumetric intersection of at least one such frustum caused by a tile in each of the previous layers, this tile is potentially visible.

We use this observation to formulate a conservative algorithm, which starts by classifying the tiles. A tile completely filled with samples is classified as *closed*, whereas a tile is *open* if it is empty or partially filled with samples. A disocclusion occurs if the viewpoint

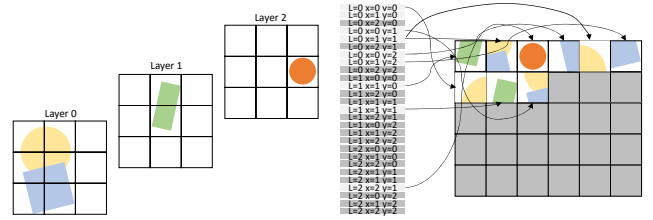


Fig. 4. (left) The scene is partitioned into three layers with 3×3 tiles per layer. (right) Page table and tile buffer corresponding to the scene.

is moved such that the scene behind a closed tile becomes visible through an open tile. Therefore, we inspect all open tiles and classify their edges with adjacent tiles in the same layer. The resulting edges patterns can be classified into six cases (Figure 3) corresponding to *disocclusion frusta* with characteristic shapes.

Each of the six shapes is chosen to have minimal volume. Only edges between an open and a closed tile generate a sloped pyramid surface. Edges between two open tiles need not be included in the frustum, since the corresponding disoccluded areas are already handled by the tile’s neighbors. Hence, the trivial sixth case of an open tile surrounded by four open tiles leads to a degenerate frustum that only covers the tile itself.

We make use of the fact that the disocclusion information of all tiles of all layers can be updated in parallel through a scattering pass without enforcing a particular order of traversing the scene. We must only ensure that disocclusion information that affects a tile is stored in a way that it can be easily collected in a final gathering pass for the classification of the tile.

Our method is fast enough to allow for online PVS computation at realistic frame rates (Figure 17). Thus, static and dynamic scenes alike can be handled naively by recomputing the PVS each frame. Alternatively, dynamic scenes with a small number of animated primitives can be handled naively by marking the dynamic primitives as visible and limiting the PVS to the static part of the scene, allowing reuse of the PVS over multiple frames (see Table 1). We propose a more elaborate refinement of our method tailored towards PVS recycling for dynamic scenes in Section 6.

3.3 Volumetric filtering of visibility

To include unreliable triangles that have evaded detection, we add a volumetric postprocessing filter. The filter is designed to deal with various types of poorly conditioned geometry. Our volumetric low-pass filter provides a robust tool for suppressing most of the problems that stem from geometric quality that we have encountered. While we apply it to potentially visible set computation in this paper, other algorithms operating on rasterized geometry can benefit from a similar method.

The filter spreads disocclusion to a small neighborhood in world coordinates. First, we collect all directly visible triangles, as indicated via triangle IDs already available from the disocclusion buffer in an initial PVS. Second, we set up a regular grid space. The grid was defined in world space, but a grid in normalized device coordinates works as well. Third, we iterate over all vertices of the visible triangles and mark a grid cell if it contains a vertex. Fourth, we



Fig. 5. Test scenes from left to right, with triangle count in parentheses: Tree Sponza (21.9 M) [Meinl et al. 2022], Bistro Exterior (3.1 M) [Amazon Lumberyard 2017], Viking Village (6.5 M) [Unity Technologies 2022], Big City (31.4 M).

iterate over all vertices of the—so far—invisible triangles and add a triangle to the PVS if any of its vertices is contained in a visible grid cell. We will demonstrate in Section 5 that this filter suppresses nearly all false negatives.

3.4 Viewcell definition

Before the actual PVS computation starts, clipping of large portions of the scene is already taken care of by hardware rasterization filling the disocclusion buffer. This rasterization requires specifying a conventional view frustum with a pinhole camera model. In contrast, computing a PVS requires first defining a viewcell. The viewcell can be seen as a generalization of a camera specification. It defines a (usually convex) subspace of 5D ray-space that contains all eligible viewpoints and view directions. The viewcell determines the range of possible viewpoints for which the PVS is valid. The disocclusion buffer must contain all portions of the scene that can be reached from any ray contained in the viewcell.

To fulfill this requirement, we adopt a method similar to the one described by Voglreiter et al. [Voglreiter et al. 2023], who adjust the camera and increase the subtended angle of the view frustum to enable additional rotations. Voglreiter et al. show that, for a frustum with a field of view of 2α and a viewcell allowing lateral motion of Δ , the PVS is guaranteed to be valid for a viewpoint displacement along the optical axis by up to $\Delta/\tan \alpha$. If a scene has metric scale, it makes sense to define a speed by which the user moves. For example, with a viewcell of 30 cm radius, a walking speed of 10 km/h, and images presented at 60 Hz, the PVS will remain valid for about six frames, covering 100 ms. In practice, we find that the PVS is still valid in many cases beyond the analytically defined viewcell. Section 5 will show results for viewcells up to 60 cm radius, doubling the amount reported in previous work [Hladky et al. 2019b; Voglreiter et al. 2023].

4 Implementation

In this section, we provide selected implementation details. Our code is available on DaRUS [Künzel et al. 2025] as open source to encourage further exploration of our approach.

4.1 Disocclusion buffer generation

We empirically determined that a tile size of $M \times M$ with $M = 8$ (for 1080p) or $M = 16$ (for 1440p and 4K) works well in our experiments. Higher resolutions, such as 4K, benefit from larger tile sizes, probably because of better disocclusion buffer coherence. Smaller resolutions do not benefit from larger tile sizes, because the

average number of empty samples in the tile (not covered by any geometric primitive) becomes too high, leading to fragmentation. A similar consideration applies to the number of layers. We found that $N = 64$ layers sufficiently cover all relevant samples needed for our applications without losing too much by depth quantization. More layers provide diminishing returns in terms of improvements in image quality compared to the increase in computational costs (see Section 5).

We index into the disocclusion buffer using the quantized coordinates of a sample point $\mathbf{p} = [x, y, z]$, where x, y are in screen coordinates, and z is the linear depth between the near clip-plane z_n and the far clip-plane z_f , i.e., $z_n < z < z_f$. The corresponding tile coordinate is

$$C(\mathbf{p}) = [c_x, c_y, c_z] = \left[\lfloor x/K \rfloor, \lfloor y/K \rfloor, \left\lfloor N \frac{\log(z - z_n + 1)}{\log(z_f - z_n + 1)} \right\rfloor \right]$$

for a disocclusion buffer with N layers. In other words, the depth interval associated with a layer is linearly related to the disparity of a sample, i.e., the distance the sample travels in screen space when it is reprojected to a new viewpoint. This property makes it easy to guarantee error bounds when reprojecting the samples.

We use a standard vertex shader to invoke the fixed-function rasterizer, but our method differs from a conventional fragment pipeline in two important aspects:

First, we do not use standard raster operations (ROP) to write fragment shader results to a framebuffer attachment. Instead, the fragment shader writes its results to a shader storage buffer object (SSBO) using random access. This strategy follows the suggestion of Clarberg et al. [Clarberg and Munkberg 2014] to leverage “the capability of modern GPUs to perform unordered memory accesses from within shaders.”

Second, as proposed by Karis et al. [Karis et al. 2021], the fragment shader uses 64-bit atomic minimum operations to implement depth buffering. Each visibility sample is a 64-bit value, with the depth occupying the most significant 32 bits and the primitive ID occupying the least significant 32 bits. Consequently, an atomic comparison determines the smaller depth value and writes the entire sample back to the disocclusion buffer without a race condition. Giving up ROP lets us replace a linear framebuffer with our own, much more efficient data structure.

The memory layout of the disocclusion buffer is organized as a two-level hierarchy [Kraus and Ertl 2002], consisting of a dense page table with pointers to a sparse tile buffer (Figure 4). We index in the page table using the tile location c_x, c_y and the tile’s layer c_z . Initially, the tile buffer is empty (all fragments have an invalid

Algorithm 1. Pseudo-code for the sequence of three compute shaders used to determine the PVS

```

parallel for all tiles  $T$ 
  if  $T$  is not empty and not closed
     $S$  = shape given by  $T$  and its neighbors
     $F$  = disocclusion frustum of  $T$  and  $S$ 
    if  $F$  is degenerate
       $P$  = tile coordinate of  $T$ 
      set bit  $L$  of visibility mask of  $P$ 
    else
      add  $F$  to disocclusion frusta array  $\mathcal{A}$ 
parallel for all disocclusion frusta  $F$  in  $\mathcal{A}$ 
  parallel for all layers  $J$  from  $L$  to  $N$  in  $F$ 
    create rectangle  $R$  from intersection of  $F$  with layer  $J$ 
    add  $R$  to shared memory array  $\mathcal{R}$ 
   $S$  = parallel prefix sum of  $\mathcal{R}$  by rectangle area
  parallel for all tiles  $C$  covered in  $\mathcal{R}$ 
     $R$  = rectangle containing  $C$  in  $\mathcal{R}$  (binary search with  $S$ )
     $L$  = layer corresponding to  $R$ 
     $P$  = tile coordinates of  $C$ 
    set bit  $L$  in visibility mask of  $P$ 

parallel for all tiles  $T$ 
   $P$  = tile coordinates of  $T$ 
   $V$  = visibility mask of  $P$ 
   $L$  = layer of  $T$ 
  if all bits 0 to  $L$  are set in  $V$ 
     $T$  is disoccluded
  else
     $T$  is occluded

```

primitive ID and a depth of z_f), and the page table entries are cleared. To write a sample at position $\mathbf{p}$, the fragment shader looks up the tile address in the page table at the entry with index $C(\mathbf{p})$.

If that entry is invalid, the tile is allocated on the fly using a fast combination of atomic operations. For that purpose, the page table contains two entries: an address that is initialized with an invalid value to indicate an empty page, and a ticket. The ticket is initialized to zero and can only be drawn once (using an atomic increment). The thread drawing the ticket then acquires an empty page via atomically incrementing the page count and writes the new address to the page table. Threads that did not draw a ticket probe the page-table entry until the address becomes available (i.e., a valid address is returned). Probing relies on atomically adding zero to the address. Adding zero does not change the state of the address, but the atomic operation forces memory synchronization and minimizes the latency until the new address becomes available.

4.2 PVS generation

The objective of PVS generation is to determine the visibility (occluded or disoccluded) for each non-empty tile. To store the results of the disocclusion computation, we allocate for each non-empty tile a *visibility mask* with one bit per layer indicating if the tile is disoccluded from a given layer. Layer 1 is stored in the least significant bit. The mask is initialized to zero, indicating that all tiles are occluded

by default. Bits are set in the visibility mask using atomic operations, so all tiles can be processed in parallel without ordering or sorting. Therefore, we avoid the drawback of synchronized back-to-front traversal that limits many previous methods [Mattausch et al. 2008; Voglreiter et al. 2023]. We run a sequence of three compute shaders (Algorithm 1).

A first compute shader processes all non-empty, open tiles in parallel. The sample counter in the page table entry indicates if a non-empty tile is open or closed. We retrieve the open/closed classification of the tile’s 4-connected neighbors and use the resulting 4-bit pattern to identify the shape of the disocclusion frustum (Figure 3). We write a corresponding record describing the disocclusion frustum to a per-tile array in global memory. For a degenerate frustum (open tile with open neighbors) of a tile at layer L , we set a single bit L in the visibility mask to indicate trivial disocclusion.

A second compute shader assigns one workgroup to each disocclusion frustum. For a disocclusion frustum spawned in layer L , the workgroup’s responsibility is to set the disocclusion bit L in the visibility masks of all tiles contained in the frustum. The frustum is intersected with all subsequent layers $i \in \{L + 1, \dots, N\}$. Every intersection with subsequent layers yields a rectangle of increasing size (i.e., number of tiles covered). The rectangle coordinates are stored in an array in shared memory (Figure 6). We run a prefix scan over the rectangle areas to obtain a searchable index into the array. For load balancing, the total number of tiles covered by the disocclusion frustum is distributed equally between the threads of the workgroup. Then, each thread uses binary search in the index to find the tile location it is responsible for and marks the corresponding entry in the tile’s visibility mask. The page table location of a tile is implicitly known from the rectangle index and the relative tile position inside the rectangle.

We obtain visibility masks that store if a tile in layer L is disoccluded in all previous layers $j \in \{1, \dots, L - 1\}$. To determine PVS inclusion, a bit operation on the tile’s visibility mask checks whether all bits in previous layers are set. This query operation is performed in a third compute shader to obtain the final PVS.

5 Results

In this section, we present evaluation results for a set of test scenes and comparisons with state-of-the-art methods. Our results were collected on a desktop computer (CPU: Intel i7-13700T with 64 GB RAM, GPU: NVidia GeForce RTX 4090, OS: Windows 11). In all experiments, the horizontal field of view of the target images was set to 60°. An extended field of view allows the PVS to cover translation and rotation (see Section 3.4). We use an extended field of view of 71°, 90° and 110° for view cell sizes of 10 cm, 30 cm and 60 cm respectively. Unless indicated separately, all experiments use a 30 cm viewcell with a 90° extended field of view. This matches the largest setup evaluated by Voglreiter et al. [Voglreiter et al. 2023]. We compared three target frame resolutions, 1080p (1920×1080), 1440p (2560×1440) and 4K (3840×2160).

The test scenes used in our evaluation are shown in Figure 5. Big City is challenging because of its size (31.7 M polygons), while the modified Tree Sponza scene (21.9 M polygons) [Meinl et al. 2022] is even more challenging because it contains a tree with high depth

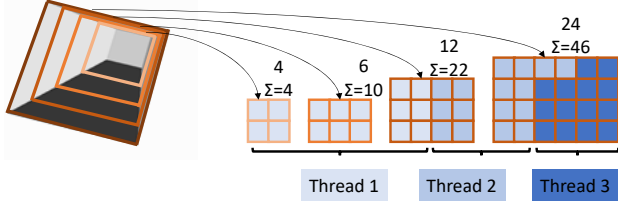


Fig. 6. (left) Intersecting a disocclusion frustum with subsequent layers produces one rectangle per layer (four rectangles in this example). (right) A prefix sum of the rectangle sizes determines that 46 tiles must be marked as disoccluded. The work is evenly distributed across a workgroup consisting of three threads, as indicated by shades of blue.

complexity resulting from very small polygons forming the foliage. In each scene, an animated camera path (600 frames) was used to generate a reproducible image sequence. The paths were chosen to represent how a human would move through the scene at a speed of about 10 km/h, taking multiple turns. The scenes have a size (bounding box diagonal) in a range of 48–1513 m. Measurements were averaged over all frames. The grid cell size of the volumetric filter grid was set to 1/1000 of the diagonal of the scene’s bounding box, with a minimum of 50 cm. Table 1 shows the average number of frames for which the PVS remained valid in our experiments, because the camera stayed within the viewcell of the given size.

5.1 Disocclusion buffer evaluation

Figure 8 shows the memory consumed to generate the disocclusion buffer (without PVS computation or shading), and Figure 9 shows the runtime measurements. The disocclusion buffer consumes between 300 and 800 MB, depending on the resolution. Otherwise, the size of the disocclusion buffer is rather insensitive to the geometric scene complexity (Figure 10). The disocclusion buffer for scenes with >30 M triangles (Tree Sponza, Big City) has a similar memory footprint as for scenes with 3 M triangles (Bistro Exterior [Amazon Lumberyard 2017]) or 6 M triangles (Viking Village [Unity Technologies 2022]). The generation times of the disocclusion buffer for small to medium scenes range from 2–4 ms (Figure 11) and peak at 7 ms for Big City at 4K resolution. PVS computation after the disocclusion buffer has been built never exceeds 0.5 ms. Regardless of the scene, the standard deviation of the disocclusion buffer generation followed by PVS computation varies between $\sigma = 0.2$ ms for 1080p resolution and $\sigma = 0.9$ ms for 4K. We did not notice any scene- or geometry-induced congestion of the disocclusion buffer generation.

To investigate the worst case, we placed the camera in dense alpha-cutout foliage. In this case, pixel-thin occluders fill a significant portion of the available layers from the near plane up to some distance. We measured a four-fold increase of overall PVS computation time in such a case. We emphasize that PVS computation in such a scenario is questionable at best, because these occlusions are extremely sensitive to camera displacements. We recommend disabling PVS computation for such pathological objects.

Furthermore, we compared using 32, 64 and 128 layers for our disocclusion buffer. The memory requirements (Figure 13), generation time (Figure 14) and tile count (Figure 14) for these layer numbers all

show an increase that is sublinear relative to the layer number. This fact makes it inexpensive to have more layers in the disocclusion buffer, leaving room for a better scene approximation if desired. A larger number of layers will retain more details of the scene, at the expense of higher memory and computation requirements. In our experiments, we settled for 64 layers as the best compromise between approximation quality and cost.

For comparison, we collected data from three data structures that could be used as an alternative to our disocclusion buffer:

Dense multi-layer buffer. A dense buffer with the same number of layers as the disocclusion buffer was implemented as an array texture. As can be expected, it has excessive memory consumption (up to 12 GB) and generation times (30–167 ms).

Sparse k -buffer. We tested a k -buffer that allocates memory separately for each pixel location. A traditional k -buffer with the k -closest fragments at arbitrary depth would require collecting disocclusions at a potentially infinite number of depths rather than in a finite number of layers. Therefore, we made the k -buffer compatible with our disocclusion algorithm by using the same fill rules for layered depth buffering as for our disocclusion buffer (Equation 1), i.e., the closest entry per pixel location and layer is kept. Our measurements reveal that the k -buffer requires less memory (1.1–4.5 GB) than the dense buffer and takes 8–12 ms to build. However, it could only support up to 1440p resolution. 4K resolution is too stressful and leads to memory-related shader crashes. Also, the k -buffer clearly performs worse than the disocclusion buffer in terms of memory and construction time. Moreover, we show below that determining disocclusions for individual pixels in the k -buffer rather than entire tiles in the disocclusion buffer is dramatically slower.

Single-layer buffer. As a lower bound, we implemented a dense single-layer buffer that stores only a single visible sample per view ray, i.e., a conventional visibility buffer. For the large scenes, it takes 1–2 ms to generate, making its generation roughly twice as fast as our disocclusion buffer. Depth peeling, which relies on rasterizing the scene repeatedly, will consume a multiple of this time, making it unattractive for fast visibility computation.

5.2 PVS evaluation

For the PVS evaluation, we compare our method with Trim Regions (TR), the previously fastest method for from-region PVS computation, using the original code provided by the authors [Voglreiter et al. 2023]. TR uses a two-level hierarchy for front-to-back sorting of fragments; the first level uses octree traversal, and the second level sorts fragments in a dense local k -buffer. The TR method propagates occlusion via an extension of occluder shrinking [Décoret et al. 2003; Wonka et al. 2000]. In that sense, we compare an occlusion approach (TR) to a disocclusion approach (ours).

In addition, we also investigate the effect of volumetric filtering. The purpose of volumetric filtering is to harden the PVS generation against spurious geometry via a mechanism that can essentially be seen as a spatial low-pass filter. We expect to trade some overestimation (more false positives in the PVS) for a substantial reduction of false negatives. We measured our method twice, once with volumetric filtering and once without.

Table 1. Average number of frames for which the PVS remains valid, as the camera stays within the viewcell.

Scene	Bistro	Tree	Viking	Bistro	Tree	Viking
	Exterior	Sponza	Village	Exterior	Sponza	Village
Viewcell	30 cm	30 cm	10 cm	30 cm	30 cm	10 cm
Resolution	1080p	1080p	1080p	4K	4K	4K
Valid frames	5	6	4	6	7	8

As a further test for the efficiency of our method, we introduce an “upsampling” configuration, where the disocclusion buffer is computed at 1080p, but the final images are rendered at 4K. This configuration combines the fast computation available at the lower resolution with image synthesis at a very high resolution. Like volumetric filtering, upsampling can be interpreted as a spatial low-pass filter, and we expect increased overestimation of the PVS.

In Figure 17, we report on the runtime of our PVS generation. It ranges from about 1.3 ms (Bistro, 1080p, 10 cm viewcell) to 8.9 ms (Big City, 4K, 60 cm viewcell). The reported runtime includes the cost of our volumetric filtering. The volumetric filtering cost depends largely on the scene’s primitive count and ranges from about 0.2 ms to 1.3 ms. The disocclusion buffer outperforms TR on average by a factor of six. Generating the PVS with the k -buffer is equal or worse than TR and much slower than the disocclusion buffer.

We state the PVS size as a fraction of the primitive count of the entire scene to a ground-truth PVS. As in previous work [Hladky et al. 2019a; Kravec et al. 2023; Voglreiter et al. 2023], we approximate ground truth by dense sampling of 576 viewpoints uniformly distributed inside the viewcell. For the disocclusion buffer, a scene primitive is considered visible if its primitive ID is contained in at least one tile marked as disoccluded in the PVS. The ground-truth PVS is typically 0.5–7% of the scene size.

Figure 18 shows the *false positive geometry rate*, i.e., the number of primitives our algorithm incorrectly reports as belonging to the PVS, divided by the size of the ground truth PVS. This rate varies with the scene and lies in the range of 97%–199% with volumetric filtering and 44%–192% without volumetric filtering. TR has a rate in the range 90%–485%, making our approach about 2–5× better than TR (except for Tree Sponza, where both methods are similar). Our upsampling method has a rate of 101%–325%, which is still substantially better than TR in most cases.

The effect of false negatives is reported in Figure 19 as *pixel errors* [Nirenstein and Blake 2004], i.e., the average number of pixels per frame for which the wrong depth or primitive ID is present in the visibility buffer. This number is reported as a fraction of the screen resolution. With volumetric filtering, the pixel error is close to zero (less than 10 wrong pixels per frame) in all scenes except Big City (which still has <300 wrong pixels at 4K). These results consistently outperform TR by a substantial margin.

Slithery triangles typically occur in distant, high-poly objects (Figure 7). It is important to note that a pixel containing a slithery triangle usually does not result in a false negative. False negatives typically arise from pixels with multiple slithery triangles in close proximity that fight for depth or coverage. This problem is a consequence of ill-conditioned geometry and frequently visible as aliasing

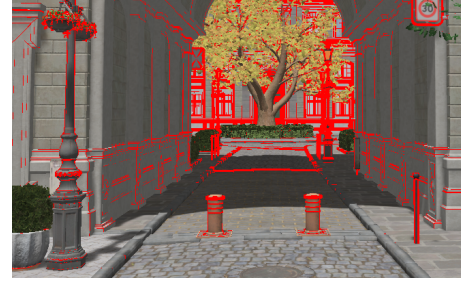


Fig. 7. Tiny or slithery triangles have at least one edge smaller than one pixel in screen space. Pixels that contain at least one such triangle are colored red.

in a conventional rendering. Our volumetric filtering mitigates the impact of slithery triangles.

For full disclosure, we also report the false positive primitive rate in Figure 20. This rate depends mainly on the ability of the filter to catch poorly conditioned geometry, which varies greatly with scene and viewpoint. However, visual inspection confirms that the image quality is not affected by these variations. To verify this observation, we also compared differences in image quality between image sequences rendered from our PVS and the ground-truth PVS, where SSIM [Wang et al. 2004] indicated no measurable errors.

6 Conclusions and future work

The disocclusion buffer computes a from-region PVS at very high speed by combining a single rasterization pass with a novel intermediate scene representation. It operates on a sparse, layered representation of the scene with quantized depth. In this representation, we invert the traditional PVS problem formulation and explicitly compute disocclusion rather than occlusion. The new problem formulation can be solved much more efficiently. Determining visibility on this representation can be done in compute shaders without any scene sorting or costly synchronization. At the same time, sparse allocation allows for a compact memory footprint, even at 4K resolution. In addition, we present a volumetric filter that effectively suppresses pixel errors resulting from rasterizing poorly conditioned geometry during PVS computation.

We see multiple avenues for future work. The rasterization pass could be replaced by a compute shader as in Nanite [Karis et al. 2021]. Layer density could be chosen adaptively to the distribution of objects in the scene. Additionally, our approach allows to change tile size and layer count in each PVS generation cycle. A histogram could be used to automate these changes. Another opportunity lies in exploiting temporal coherence between successive PVS computations. Memory footprint can be reduced by storing depth and primitive ID relative to each layer. A 32-bit visibility sample with 16-bit depth per layer and 16-bit primitive ID per layer seems feasible. Finally, we envision an extension of our method to include animated objects in the PVS without recomputing the PVS every frame. Individual objects could be rasterized into their own sparse disocclusion buffer in each frame at a fraction of the cost. The disocclusion buffer storing the static objects of the scene remains valid, and only the PVS needs to be recomputed. This approach would be

significantly cheaper, as the PVS computation is much faster than the disocclusion buffer generation. Furthermore, the PVS computation could be adapted to allow partial updates of the visibility volumes.

Acknowledgments

We thank Bernhard Kerbl for Big City. This work was supported by the Alexander von Humboldt Foundation funded by the German Federal Ministry of Research, Technology and Space, the German Research Foundation *DFG*—Project-ID 251654672—TRR 161 and Germany’s Excellence Strategy—EXC 2120/1—390831618, as well as the Austrian Science Fund *FWF* (grant I6663).

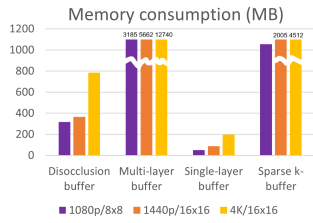


Fig. 8. Memory comparison of disocclusion buffer, dense multi-layer buffer, sparse k -buffer and single-layer buffer (only frontmost samples). The disocclusion buffer needs 4–6× more than a single layer, but stays <800 MB for 4K.

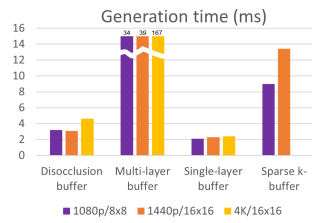


Fig. 9. Generating the disocclusion buffer takes about 3–5 ms, which is 1.5–2× longer than generating a single-layer buffer. Generating a multi-layer buffer or k -buffer takes much longer.

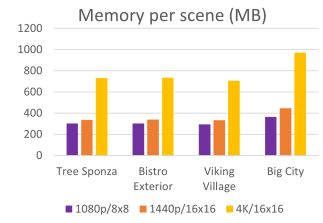


Fig. 10. Memory consumption of the disocclusion buffer is rather insensitive to the geometric complexity of the scene.

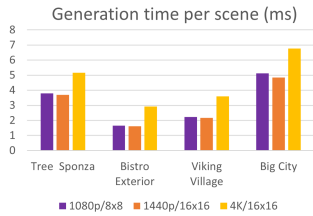


Fig. 11. Smaller scenes need 2–4 ms to generate the disocclusion buffer. Note that, depending on the frame number for which the PVS remains valid, even longer generation times can be amortized.

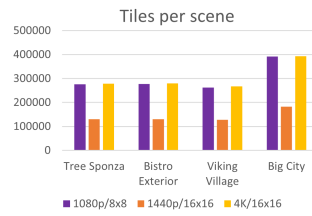


Fig. 12. Tiles per scene depend on the ratio of tile size and the screen resolution. 1080p and 4K have the same ratio, 1440p has a higher ratio, leading to a smaller tile count.

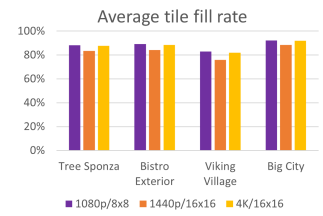


Fig. 13. The tile fill rate, i.e., the number of filled samples per tile is overall very high (about 76%–92%) and does not vary much. Less than every fourth sample is left empty.

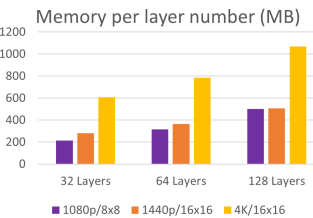


Fig. 14. Memory requirements of the disocclusion buffer are sub-linear in the layer number. Doubling the layers increases memory only by 29%–58%.

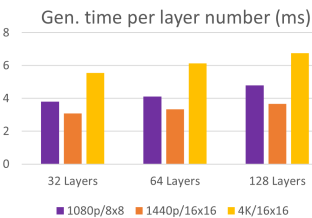


Fig. 15. Generation times of the disocclusion buffer are sub-linear in layer number. Doubling the layers increases generation time by 8%–16%.

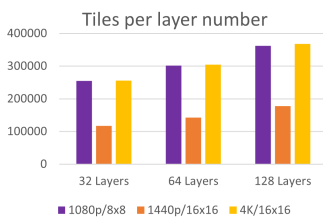


Fig. 16. The tile number is sub-linear in layer number. Doubling the layers increases the number of tiles by 18%–25%.

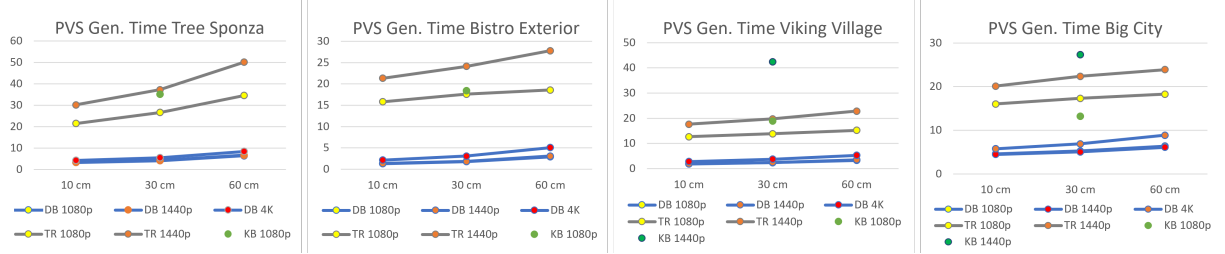


Fig. 17. PVS generation times in ms, grouped by scene and organized by viewcell size (10, 30 and 60 cm). We compare our disocclusion buffer (DB) to the state-of-the-art method Trim Regions (TR). TR could not support 4K resolution. A k -buffer (KB, green dots) is clearly slower than our tiled approach.

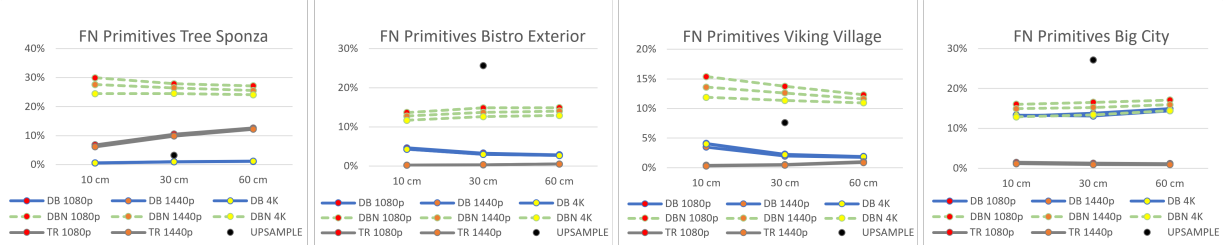


Fig. 18. False positive primitive rate in the PVS: We report the fraction of primitives unnecessarily included in our PVS compared to the ground truth PVS, averaged over all scenes and animation paths. We compare the disocclusion buffer with volumetric filter (DB, solid blue lines) and without volumetric filtering (DBN, dashed lines), the Trim Region method (TR) and the disocclusion buffer computed at 1080p used for rendering images at 4K (UPSAMPLE, black dots).

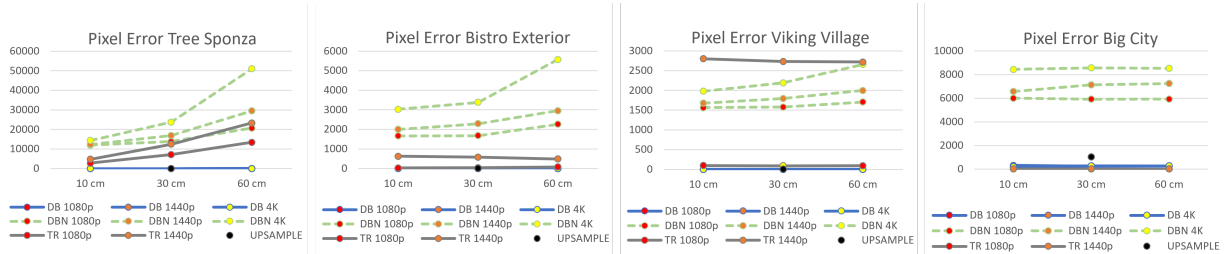


Fig. 19. Pixel errors: We report the number of incorrect pixels observed in all scenes and animations paths. We compare the disocclusion buffer with volumetric filter (DB, solid blue lines) and without volumetric filtering (DBN, dashed lines), the Trim Region method (TR) and the disocclusion buffer computed at 1080p used for rendering images at 4K (UPSAMPLE, black dots). With volumetric filtering, the disocclusion buffer consistently has near-zero (<10) pixels errors, except for slightly higher errors in Big City. This is also true for disocclusion buffer with upsampling (UPSAMPLE).

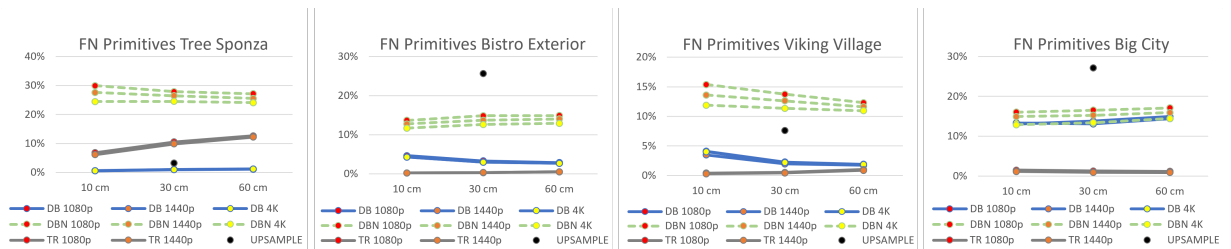


Fig. 20. For completeness, we list false negative primitives rates, which are generally higher than observed pixel errors. Note that missed primitives are usually caused by poorly conditioned geometry (tiny or slithery triangles) and has near-zero effect on image quality (see Figure 19).

References

- M Airey, J Rohlf, and Frederick Brooks Jr. 1990. Towards Image Realism with Interactive Update Rates in Complex Virtual Building Environments. *ACM SIGGRAPH Computer Graphics* 24 (1990), 41–50. doi:10.1145/91385.91416
- Amazon Lumberyard. 2017. Amazon Lumberyard Bistro, Open Research Content Archive (ORCA). <http://developer.nvidia.com/orca/amazon-lumberyard-bistro> Last visited: 2025-09-25.
- Louis Bavoil, Steven P. Callahan, Aaron Lefohn, João L. D. Comba, and Cláudio T. Silva. 2007. Multi-fragment Effects on the GPU Using the k-buffer. In *Proceedings of I3D*. Association for Computing Machinery, New York, 97–104. doi:10.1145/1230100.1230117
- J. Bittner, V. Havran, and P. Slavik. 1998. Hierarchical Visibility Culling with Occlusion Trees. In *Proceedings. Computer Graphics International (Cat. No.98EX149)*. IEEE Comput. Soc, Los Alamitos, CA, USA, 207–219. doi:10.1109/CGI.1998.694268
- Jiri Bittner, Oliver Mattausch, Peter Wonka, Vlastimil Havran, and Michael Wimmer. 2009. Adaptive Global Visibility Sampling. In *ACM SIGGRAPH '09*, Vol. 28(3). ACM Press, New York, NY, USA, 1. doi:10.1145/1576246.1531400
- Jiri Bittner, Michael Wimmer, Harald Piringer, and Werner Purgathofer. 2004. Coherent Hierarchical Culling: Hardware Occlusion Queries Made Useful. *Computer Graphics Forum* 23, 3 (9 2004), 615–624.
- Jiri Bittner and Peter Wonka. 2003. Visibility in Computer Graphics. *Environment and Planning B: Planning and Design* 30, 5 (2003), 729–755. doi:10.1068/b2957
- Christopher A Burns and Warren A Hunt. 2013. The Visibility Buffer: A Cache-Friendly Approach to Deferred Shading. *Journal of Computer Graphics Techniques* 2 (2013), 55–69. Issue 2. <http://jcgf.org/published/0002/02/04/>
- Loren Carpenter. 1984. The A-buffer, an Antialiased Hidden Surface Method. In *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques*. ACM, New York, NY, USA, 103–108. doi:10.1145/800031.808585
- Anish Chandak, Lakulish Antani, Micah Taylor, and Dinesh Manocha. 2009. FastV: From-point Visibility Culling on Complex Models. In *Proc. of the 20th Eurographics Conference on Rendering (EGSR'09)*. Eurographics Association, Eindhoven, Netherlands, 1237–1246. doi:10.1111/j.1467-8659.2009.01501.x
- Petrik Clarberg and Jacob Munkberg. 2014. Deep Shading Buffers on Commodity GPUs. *ACM Transactions on Graphics* 33, 6 (2014), 227:1–227:12. doi:10.1145/2661229.2661245
- D. Cohen-Or, Y.L. Chrysanthou, C.T. Silva, and F. Durand. 2003. A Survey of Visibility for Walkthrough Applications. *IEEE Transactions on Visualization and Computer Graphics* 9, 3 (2003), 412–431. doi:10.1109/TVCG.2003.1207447
- Xavier Décoret, Gilles Debunne, and François Sillion. 2003. Erosion Based Visibility Preprocessing. In *Proc. of the 14th Eurographics Workshop on Rendering (EGRW '03)*. Eurographics Association, Aire-la-Ville, Switzerland, Switzerland, 281–288.
- Frédo Durand, George Drettakis, and Claude Puech. 2002. The 3D visibility complex. *ACM Transactions on Graphics* 21, 2 (2002), 176–206. doi:10.1145/508357.508362
- Frédo Durand, George Drettakis, Joëlle Thollot, and Claude Puech. 2000. Conservative Visibility Preprocessing Using Extended Projections. In *Proceedings of the 27th annual conference on Computer graphics and interactive techniques - SIGGRAPH '00*. ACM Press, New York, NY, USA, 239–248. doi:10.1145/344779.344891
- Epic Games. 2022. Precomputed Visibility Volumes. <https://docs.unrealengine.com/5.1/en-US/precomputed-visibility-volumes-in-unreal-engine/> Last visited: 2022-12-21.
- C. Everitt. 2001. Interactive Order-Independent Transparency. white paper, Nvidia.
- Ned Greene, Michael Kass, and Gavin Miller. 1993. Hierarchical Z-buffer Visibility. In *Proceedings of the 20th annual conference on Computer graphics and interactive techniques - SIGGRAPH '93*. ACM Press, New York, NY, USA, 231–238. doi:10.1145/166117.166147
- Ulrich Haar and Sebastian Aaltonen. 2015. GPU-Driven Rendering Pipelines. *SIGGRAPH Course: Advances in Real-Time Rendering in Games*.
- Jon Hasselgren, Magnus Andersson, and Tomas Akenine-Möller. 2016. Masked Software Occlusion Culling. In *Eurographics/ ACM SIGGRAPH Symposium on High Performance Graphics*, Ulf Assarsson and Warren Hunt (Eds.). The Eurographics Association, Eindhoven, Netherlands, 23–31. doi:10.2312/hpg.20161189
- Jozef Hladky, Hans-Peter Seidel, and Markus Steinberger. 2019a. The Camera Offset Space: Real-Time Potentially Visible Set Computations for Streaming Rendering. *ACM Transactions on Graphics* 38, 6, Article 231 (2019), 14 pages. doi:10.1145/3355089.3355330
- J. Hladky, H. P. Seidel, and M. Steinberger. 2019b. Tessellated Shading Streaming. *Computer Graphics Forum* 38, 4 (2019), 171–182. doi:10.1111/cgf.13780
- Jozef Hladky, Michael Stengel, Nicholas Vining, Bernhard Kerbl, Hans-Peter Seidel, and Markus Steinberger. 2022. QuadStream: A Quad-Based Scene Streaming Architecture for Novel Viewpoint Reconstruction. *ACM Transactions on Graphics* 41, 6, Article 233 (2022), 13 pages. doi:10.1145/3550454.3555524
- T. Hudson, D. Manocha, J. Cohen, M. Lin, K. Hoff, and H. Zhang. 1997. Accelerated Occlusion Culling Using Shadow Frusta. In *Proceedings of the Annual Symposium on Computational Geometry*. ACM, New York, NY, USA, 1–10. doi:10.1145/262839.262847
- Brian Karis, Rune Stubbe, and Graham Wihlidal. 2021. A Deep Dive into Nanite Virtualized Geometry. *SIGGRAPH Course on Advances in Real-Time Rendering in Games*. https://advances.realtimerendering.com/s2021/Karis_Nanite_SIGGRAPH_Advances_2021_final.pdf Last visited: 2025-09-25.
- Janghun Kim and Sungkil Lee. 2023. Potentially Visible Hidden-Volume Rendering for Multi-View Warping. *ACM Transactions on Graphics* 42, 4, Article 86 (2023), 11 pages. doi:10.1145/3592108
- Thomas Koch and Michael Wimmer. 2021. Guided Visibility Sampling++. *Proceedings of Computer Graphics and Interactive Techniques* 4, 1, Article 4 (2021), 16 pages. doi:10.1145/3451266
- Vladlen Koltun, Yiorgos Chrysanthou, and Daniel Cohen-Or. 2000. Virtual Occluders: An Efficient Intermediate PVS Representation. In *Rendering Techniques 2000*, Bernard Péroche and Holly Rushmeier (Eds.). Springer, Vienna, Vienna, Austria, 59–70. doi:10.1007/978-3-7091-6303-0_6
- Martin Kraus and Thomas Ertl. 2002. Adaptive Texture Maps. In *SIGGRAPH/Eurographics Workshop on Graphics Hardware*, Thomas Ertl, Wolfgang Heidrich, and Michael Doggett (Eds.). The Eurographics Association, Goslar, Germany, 7–15. doi:10.2312/EGGH/EGGH02/007-015
- Jaroslav Kravec, Martin Kácerik, and Jiri Bittner. 2023. PVLI: Potentially Visible Layered Image for Real-Time Ray Tracing. *The Visual Computer* 39, 8 (2023), 3359–3372. doi:10.1007/s00371-023-03007-5
- Sebastian Künzel, Sergej Geringer, Quynh Quang Ngo, Philip Voglreiter, Daniel Weiskopf, and Dieter Schmalstieg. 2025. Supplemental Material for “Potentially Visible Set Generation with the Disocclusion Buffer”. doi:10.18419/DARUS-5385
- Tommer Leyvand, Olga Sorkine, and Daniel Cohen-Or. 2003. Ray Space Factorization for From-Region Visibility. In *ACM SIGGRAPH 2003 Papers on - SIGGRAPH '03*, Vol. 22(3). ACM Press, New York, NY, USA, 595. doi:10.1145/1201775.882313
- Michael Mara, Morgan McGuire, Derek Nowrouzezahrai, and David Luebke. 2016. Deep G-Buffers for Stable Global Illumination Approximation. In *Eurographics/ ACM SIGGRAPH Symposium on High Performance Graphics*, Ulf Assarsson and Warren Hunt (Eds.). The Eurographics Association, Goslar, Germany, 87–98. doi:10.2312/hpg.20161195
- Oliver Mattausch, Jiri Bittner, and Michael Wimmer. 2008. CHC++: Coherent Hierarchical Culling Revisited. *Computer Graphics Forum* 27, 2 (2008), 221–230. doi:10.1111/j.1467-8659.2008.01119.x
- Frank Meinel, Anton Kaplanyan, Katia Putica, Cristiano Siqueira, Timothy Heath, Justin Prazen, Sebastian Herholz, and Bruce Cherniak. 2022. Intel Samples Library. <https://www.intel.com/content/www/us/en/developer/topic-technology/graphics-processing-research/samples.html> Last visited: 2025-09-25.
- Joerg H. Mueller, Philip Voglreiter, Mark Dokter, Thomas Neff, Mina Makar, Markus Steinberger, and Dieter Schmalstieg. 2018. Shading Atlas Streaming. *ACM Transactions on Graphics* 37, 6, Article 199 (Dec. 2018), 16 pages. doi:10.1145/327127.3275087
- S. Nirenstein and E. Blake. 2004. Hardware accelerated visibility preprocessing using adaptive sampling. In *Proceedings of Eurographics Conference on Rendering Techniques (EGSR'04)*. The Eurographics Association, Goslar, Germany, 207–216. doi:10.2312/EGWR/EGSR04/207-216
- Voicu Popescu, Elisha Sacks, Jian Cui, and Rohan Ashok. 2024. Efficient and Robust From-Point Visibility. *IEEE Transactions on Visualization and Computer Graphics* 30, 8 (2024), 5313–5327. doi:10.1109/TVCG.2023.3291138
- Stefan Popov, Iliyan Georgiev, Philipp Slusallek, and Carsten Dachsbacher. 2013. Adaptive Quantization Visibility Caching. *Computer Graphics Forum* 32 (May 2013), 399–408. doi:10.1111/cgf.12060
- Jonathan Ragan-Kelley, Jaakko Lehtinen, Jiawen Chen, Michael Doggett, and Frédo Durand. 2011. Decoupled Sampling for Graphics Pipelines. *ACM Transactions on Graphics* 30, 3, Article 17 (2011), 17 pages. doi:10.1145/1966394.1966396
- Bernhard Reinert, Johannes Kopf, Tobias Ritschel, Eduardo Cuervo, David Chu, and Hans-Peter Seidel. 2016. Proxy-guided Image-based Rendering for Mobile Devices. *Computer Graphics Forum* 35 (2016), 353–362.
- Takafumi Saito and Tokiichiro Takahashi. 1990. Comprehensive Rendering of 3-D Shapes. *Proc. SIGGRAPH* 24, 4 (1990), 197–206. doi:10.1145/97880.97901
- Gernot Schaufner, Julie Dorsey, Xavier Décoret, and François X. Sillion. 2000. Conservative Volumetric Visibility with Occluder Fusion. In *Proceedings of SIGGRAPH*. Association for Computing Machinery/Addison-Wesley Publishing Co., New York, 229–238. doi:10.1145/344779.344886
- Yvens Rebouças Serpa and Maria Andréia Formico Rodrigues. 2019. A Draw Call-oriented Approach for Visibility of Static and Dynamic Scenes with Large Number of Triangles. *Visual Computer* 35, 4 (apr 2019), 549–563.
- Jonathan Shade, Steven Gortler, Li-wei He, and Richard Szeliski. 1998. Layered Depth Images. In *Proceedings of SIGGRAPH*. Association for Computing Machinery, New York, 231–242. doi:10.1145/280814.280882
- Zhipeng Tan, Yongxiang Zhang, Fei Xia, and Fei Ling. 2025. Differentiable Rendering based Part-Aware Occlusion Proxy Generation. *Computer Graphics Forum* 44, 2 (2025), e70077. doi:10.1111/cgf.70077
- Seth J. Teller and Carlo H. Séquin. 1991. Visibility Preprocessing for Interactive Walkthroughs. In *Proceedings of the 18th annual conference on Computer graphics and interactive techniques - SIGGRAPH '91*, Vol. 25(4). ACM Press, New York, NY, USA, 61–70. doi:10.1145/122718.122725

- Unity Technologies. 2022. Viking Village URP (Unity Asset). <https://assetstore.unity.com/packages/essentials/tutorial-projects/viking-village-urp-29140> Last visited: 2025-09-25.
- Andreas Alexandros Vasilakis, Konstantinos Vardis, and Georgios Papaioannou. 2020. A Survey of Multiframe Rendering. *Computer Graphics Forum* 39, 2 (2020), 623–642. doi:10.1111/cgf.14019
- Philip Voglreiter, Bernhard Kerbl, Alexander Weinrauch, Joerg Hermann Mueller, Thomas Neff, Markus Steinberger, and Dieter Schmalstieg. 2023. Trim Regions for Online Computation of From-Region Potentially Visible Sets. *ACM Transactions on Graphics* 42, 4, Article 85 (2023), 15 pages. doi:10.1145/3592434
- Zhou Wang, A.C. Bovik, H.R. Sheikh, and E.P. Simoncelli. 2004. Image Quality Assessment: from Error Visibility to Structural Similarity. *IEEE Transactions on Image Processing* 13, 4 (2004), 600–612. doi:10.1109/TIP.2003.819861
- Peter Wonka, Michael Wimmer, and Dieter Schmalstieg. 2000. Visibility Preprocessing with Occluder Fusion for Urban Walkthroughs. In *Proceedings of EUROGRAPHICS Workshop on Rendering*. Springer Vienna, Vienna, Austria, 71–82.
- Peter Wonka, Michael Wimmer, and Francois X. Sillion. 2001. Instant Visibility. *Computer Graphics Forum* 20, 3 (2001), 411–421. doi:10.1111/1467-8659.00534
- Jason C. Yang, Justin Hensley, Holger Grün, and Nicolas Thibieroz. 2010. Real-Time Concurrent Linked List Construction on the GPU. *Computer Graphics Forum* 29 (June 2010), 1297–1304. doi:10.1111/j.1467-8659.2010.01725.x